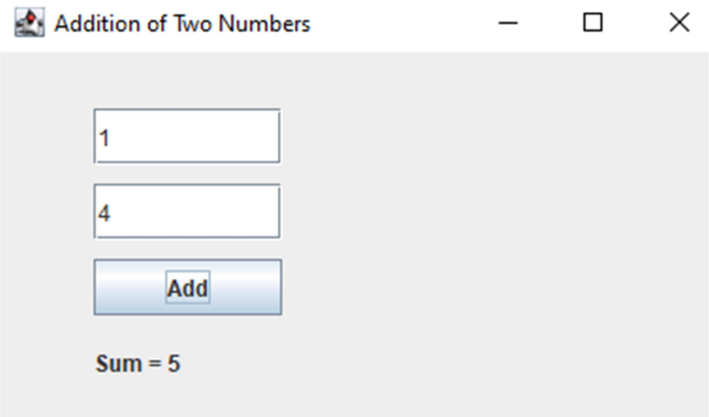
```
}public void actionPerformed(ActionEvent e){
    try { int num1 = Integer.parseInt(num1field.getText());
          int num2 = Integer.parseInt(num2field.getText());
          int sum = num1 + num2;
          resultLable.setText("Sum = "+sum);
        } catch (NumberFormatException ex){
          resultLable.setText("Please enter valid integers");
        }
    }
    public static void main(String[]args){
        new SumCalculator();
    }
}
```

Observation

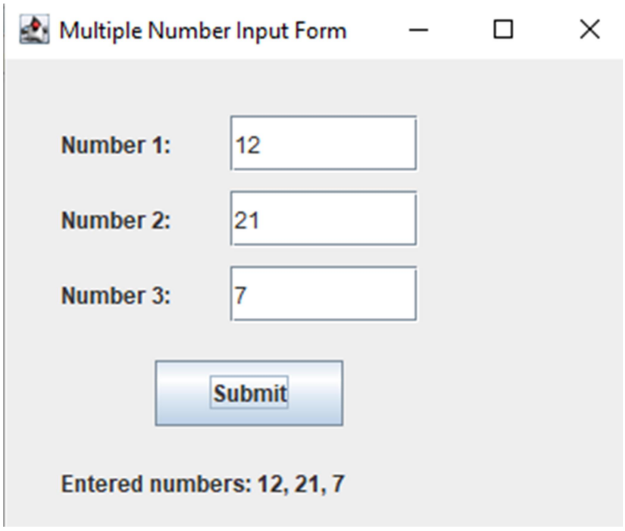


Conclusion

The lab demonstrated how to use Java Swing to create a simple GUI application that takes two numbers as input and calculates their sum. This exercise helped understand GUI components, event handling, and basic user input validation in Java.

```
    setDefaultOperation(JFrame.EXIT_ON_CLOSE);
}
public void actionPerformed(ActionEvent e){
  try{
      int a = Integer.parseInt(n1.getText());
      int b = Integer.parseInt(n2.getText());
      int c = Integer.parseInt(n3.getText());
      result.setText("Entered:"+a+","+b+","+c);
    } catch (Exception ex) {
        result.setText("Enter valid Integers!");
    }
  }
  public static void main(String[] args){
      new MultipleInputForm();
  }
}
```

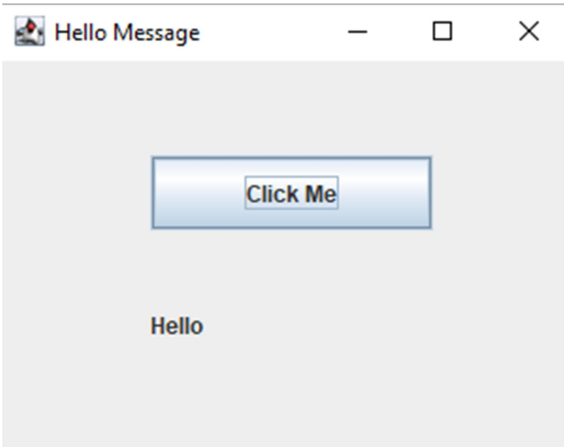
Observation



Conclusion

This lab demonstrated the creation of a form in Java Swing to accept multiple number inputs from the user. It helped understand GUI design, form handling, input validation, and event-driven programming using Swing.

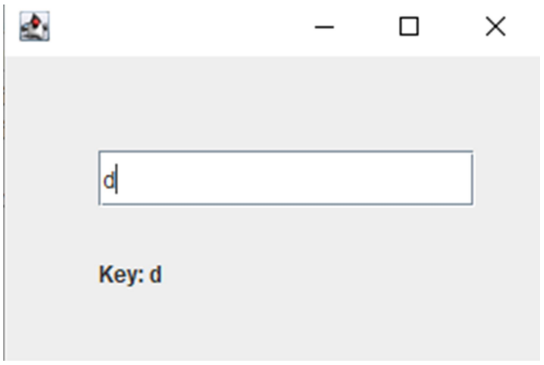
Observation



Conclusion

This experiment demonstrated how to handle button click events in Swing using ActionListener. The program successfully displays a "Hello" message when the user clicks the button, illustrating basic event-driven GUI programming.

Observation

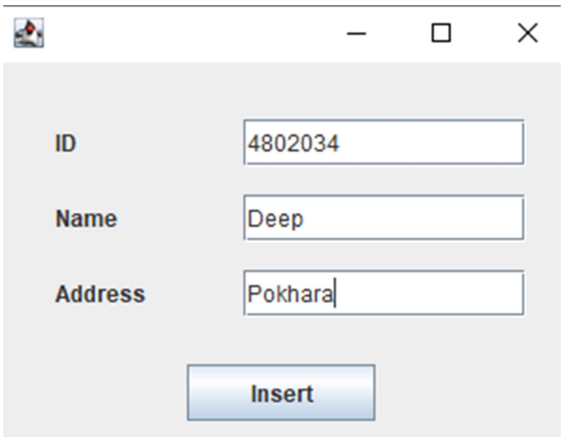


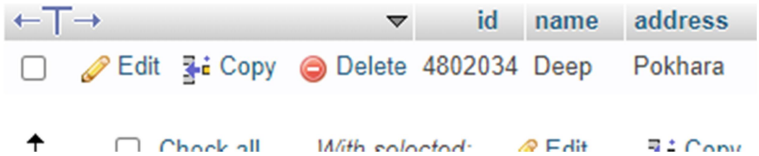
Conclusion

This experiment demonstrated how to handle keyboard events in Java Swing using the KeyAdapter class. It allowed detecting key presses in real-time and displayed the pressed key on the GUI, reinforcing understanding of event-driven programming concepts.

```
public static void main(String[] args){
    new StudentInsert();
}
}
```

Observation

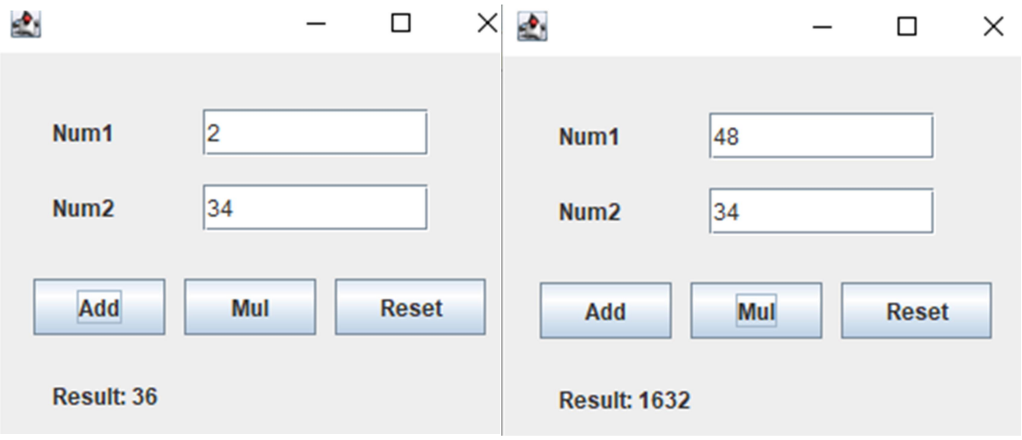




Conclusion

This lab successfully demonstrates how to collect student data using a Swing form and insert it into a MySQL database using JDBC. It provides practical understanding of GUI development, database connectivity, SQL operations, and event handling in Java.

Observation



Conclusion

This lab demonstrated how to build a simple calculator using Java Swing. It covered GUI form creation, button event handling, arithmetic operations, and resetting input fields, strengthening understanding of interactive application development.

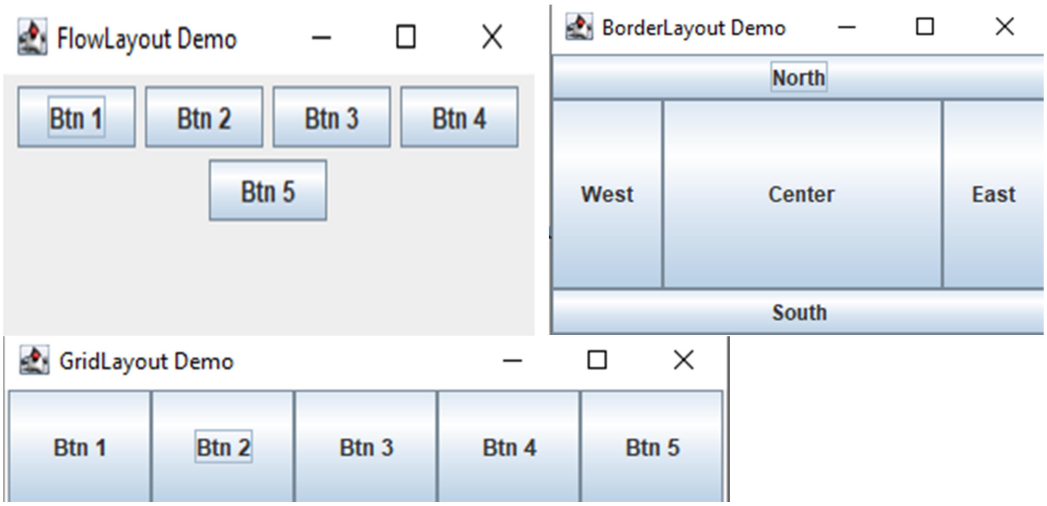
Observation

```
C:\Users\user\Desktop\pncw\BCA\6th sem\advJava>java BeanDemo
Student Name: Deep
Student Age: 22
```

Conclusion

This experiment demonstrated the creation and implementation of a JavaBean. The program showed how to define properties, apply encapsulation using getter and setter methods, and implement the bean in another class. JavaBeans make components reusable and maintainable in larger applications.

Observation



Conclusion

The experiment successfully demonstrated the use of FlowLayout, BorderLayout, and GridLayout in Java Swing. Each layout manager controls component arrangement differently, showing how Java provides flexible tools for designing graphical user interfaces.

Observation

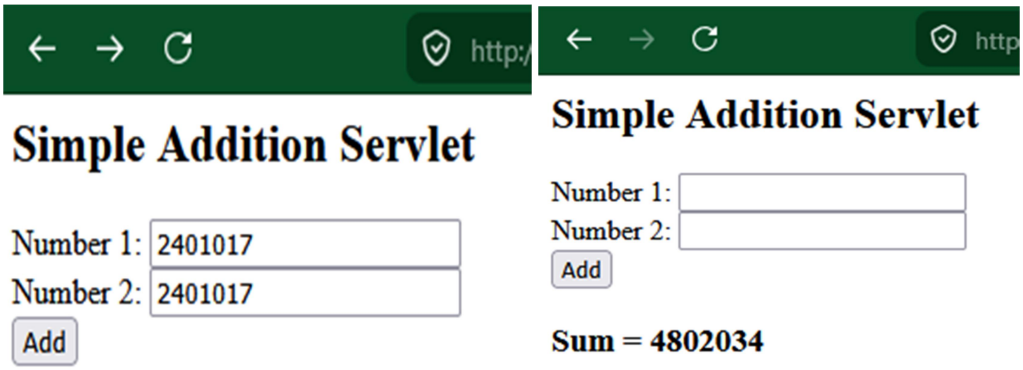
Conclusion

The experiment successfully demonstrated how to create and run a basic servlet using Java. The servlet processed an HTTP GET request and displayed a message on a web page, showing the working of Java Web components.

Webxml

```
h<web-app   xmlns="http://xmlns.jcp.org/xml/ns/javaee"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
http://xmlns.jcp.org/xml/ns/javaee/web-app_3_1.xsd"

         version="3.1">

    <servlet>

        <servlet-name>AddServlet</servlet-name>

        <servlet-class>AddServlet</servlet-class>

    </servlet>

    <servlet-mapping>

        <servlet-name>AddServlet</servlet-name>

        <url-pattern>/add</url-pattern>

    </servlet-mapping>

</web-app>
```

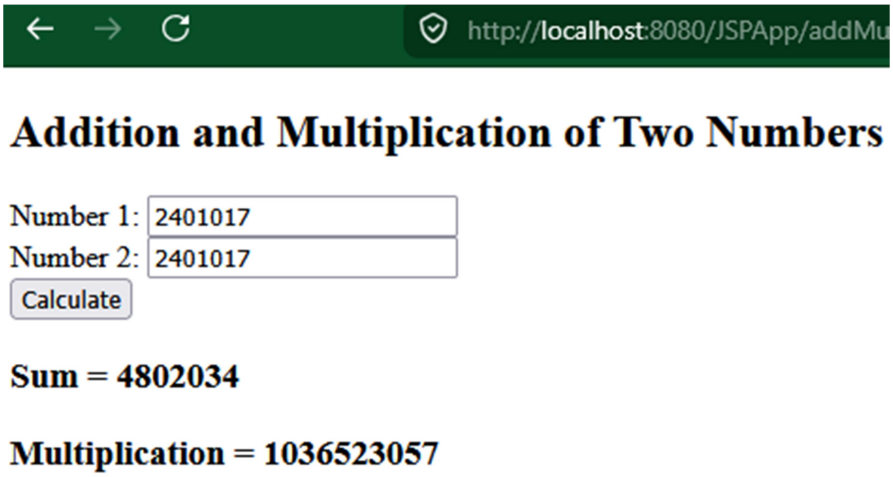
Observation



Conclusion

The lab successfully demonstrates handling user input using servlets. Java servlets can dynamically generate HTML content and process request parameters. This lab is a practical example of server-side programming using Java.

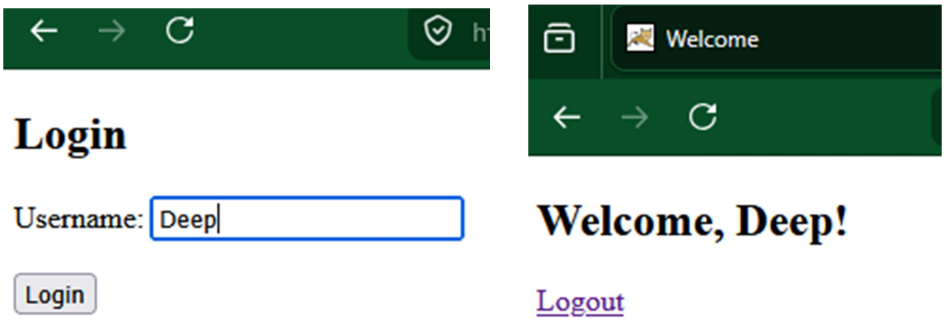
Observation



Conclusion

The lab demonstrates server-side processing using JSP. JSP allows embedding Java code in HTML for dynamic web content. Addition and multiplication operations were successfully performed, validating form handling and data processing

Observation



Conclusion

This lab demonstrates session management in JSP, allowing server-side storage of user data and maintaining state across multiple pages. The login form validates the username, stores it in session, and dynamically displays personalized content.

Observation

```
PS C:\xampp\tomcat\webapps\chat> javac *.java
PS C:\xampp\tomcat\webapps\chat> start rmiregistry
PS C:\xampp\tomcat\webapps\chat> java ChatServer
Server is ready...
Client: hello world
```

```
PS C:\xampp\tomcat\webapps\chat> java ChatClient
Enter message:
hello world
Server received: hello world
PS C:\xampp\tomcat\webapps\chat>
```

Conclusion

This lab demonstrates a basic RMI-based chat application, showing how Java RMI allows communication between objects in different JVMs. It illustrates remote method invocation, binding, and interaction between client and server in a distributed environment

Observation

```
PS C:\xampp\tomcat\webapps\RMIAdd> start rmiregistry 1099
```

```
C:\Program Files\Eclipse Adoptium\jdk-17.0.10.7-hotspot\bin\rmiregistry.exe
WARNING: A terminally deprecated method in java.lang.System has been called
WARNING: System::setSecurityManager has been called by sun.rmi.registry.RegistryImpl
WARNING: Please consider reporting this to the maintainers of sun.rmi.registry.RegistryImpl
WARNING: System::setSecurityManager will be removed in a future release
```

```
PS C:\xampp\tomcat\webapps> cd C:\xampp\tomcat\webapps\RMIAdd
PS C:\xampp\tomcat\webapps\RMIAdd> java AddServer
Addition Server is ready...
```

```
PS C:\xampp\tomcat\webapps\RMIAdd> java AddClient
Sum from server: 12
PS C:\xampp\tomcat\webapps\RMIAdd>
```

Conclusion

This lab demonstrates a simple RMI-based addition application, showing how Java RMI allows clients to invoke methods on remote servers. It illustrates client-server communication, remote method calls, and distributed processing in Java.